

# Distributed Monitoring System using Prometheus and Thanos on openSUSE Servers

Geraldhi Aditya Putra Mahayadnya

Kelompok Linux Arek Suroboyo (KLAS)

# About Me



## Geraldhi Aditya Putra Mahayadnya

Member of Kelompok Linux Arek Suroboyo (KLAS)  
Surabaya Linux User Group

Currently working as an IT System Infrastructure Engineer  
In a Private Banking Company in Indonesia

Loves Gaming on Linux with SteamDeck.  
IEM Enjoyer :D

Github: @geraldapm  
Instagram: @gerald\_apm  
Telegram: @geraldapm  
LinkedIn: linkedin.com/in/geraldapm  
Email: geraldaditya1111@gmail.com

CNCF KubeStronaut (CKA, CKAD, CKS, KCNA, KCSA),  
Cisco CCNA (Expired), MTCRE, MTCINE

# About KLAS



openSUSE<sup>®</sup>  
Asia Summit 2026  
Yogyakarta, October 3 - 4

KLAS is a Linux User Group based in Surabaya, Indonesia. It is began with a group of Linux Enthusiasts aiming for community development of Linux Adaptation.



Our Services: [s.klas.or.id](https://s.klas.or.id) & [buaya.klas.or.id](https://buaya.klas.or.id)



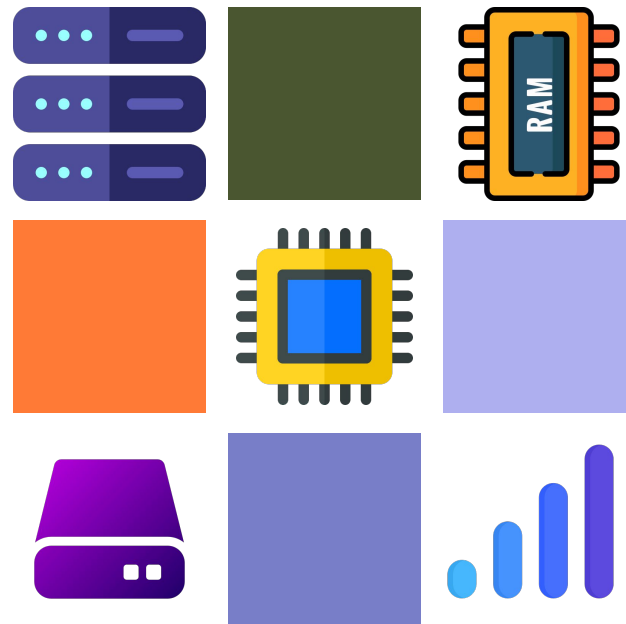


# Outlines

- Monitoring metrics on servers
- Usage of Prometheus and Node Exporter
- Challenges when the large number of servers exists and metrics retention
- Integrating Prometheus and Thanos
- Scaling the metric retentions using Object Storage solution
- Distributed monitoring implementation with demos

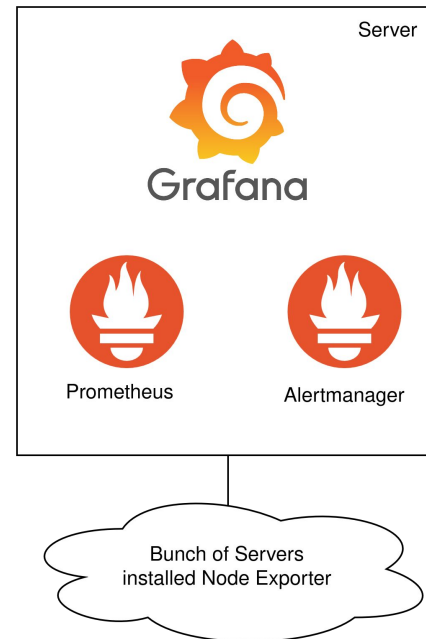
# Monitoring metrics on servers

- Metrics monitoring on servers is necessary to ensure the server and services availability on the system.
- It consists of CPU, Memory, Disk, Networking, and many more parameters to be monitored depending on system requirements.
- Necessary monitoring adjustments is required to ensure that the monitoring platform is suited to our requirements.



# Monitoring with Prometheus and Node Exporter

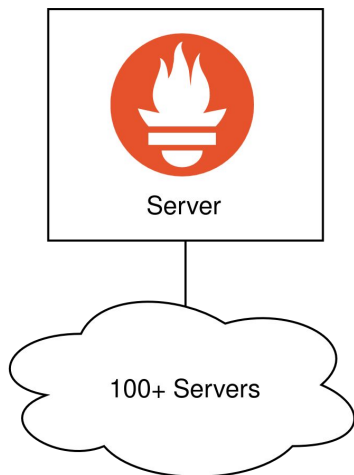
- We can use Prometheus and Node Exporter to enable monitoring on our openSUSE Servers.
- One Instance is used to serve as Prometheus Server and other servers are installed Node Exporter to scrape the OS-side metrics.
- Then we can deploy Grafana and alertmanager (usually on the same server as Prometheus) to view our metrics and enable alerting.



# Challenges

# Scaling Server vs Prometheus Instances

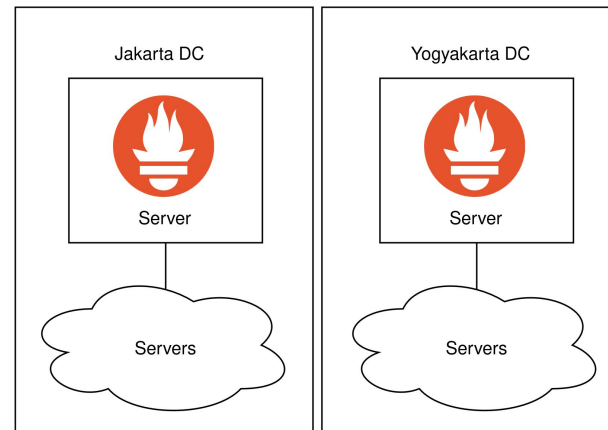
Scenario 1  
Very Large number of servers



Scale with CPU and  
Memory

Horizontal Scaling?

Scenario 2  
Multiple Data Center

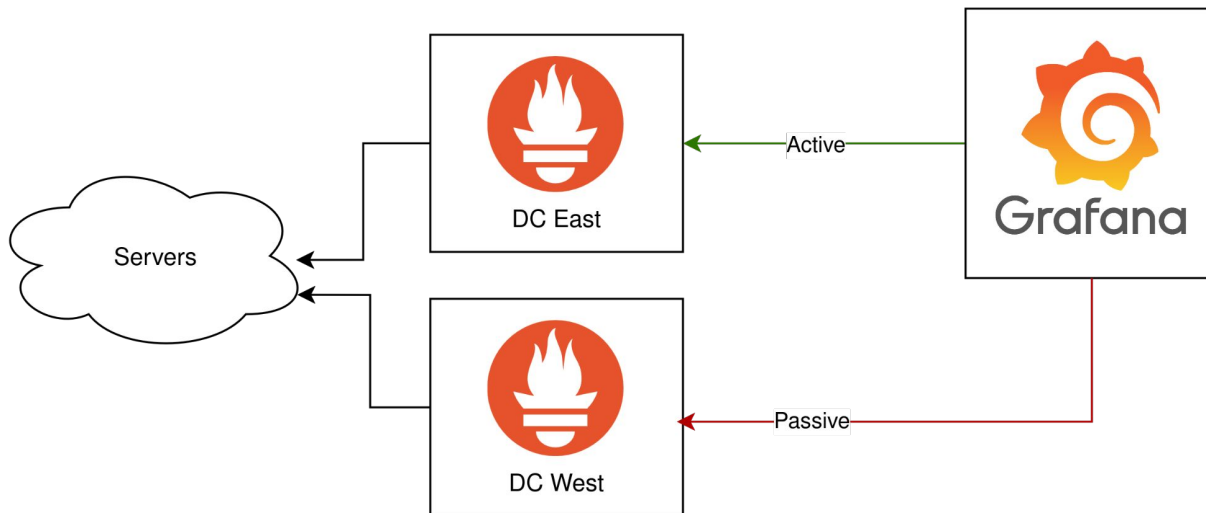


Deploying Prometheus on  
each Data Center

Centralized View?

# Prometheus Global Data View

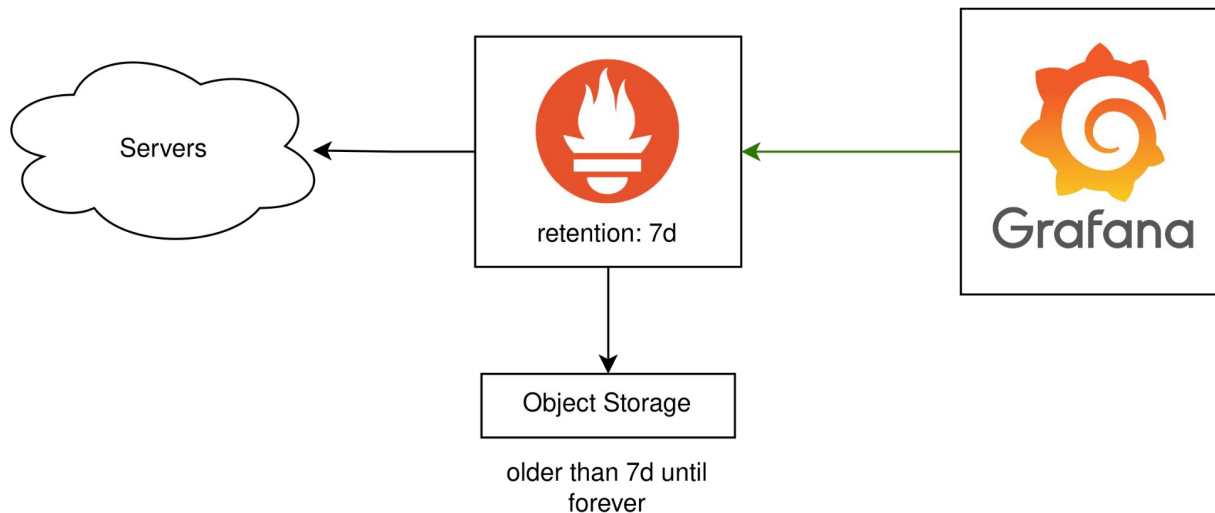
Deploying Prometheus Server with manual Data Source change on Grafana. We might have several different Dashboard for each Prometheus instances.



No Centralized endpoint for global metrics query.

# Prometheus Data Retention

Deploying Prometheus Server with longer retention requires more storage.



The need of storing prometheus metrics into Object Storage for longer metrics retention and cheaper price.



**openSUSE®**  
**Asia Summit 2026**  
Yogyakarta, October 3 - 4

# Integrating with Thanos

# Integrating with Thanos

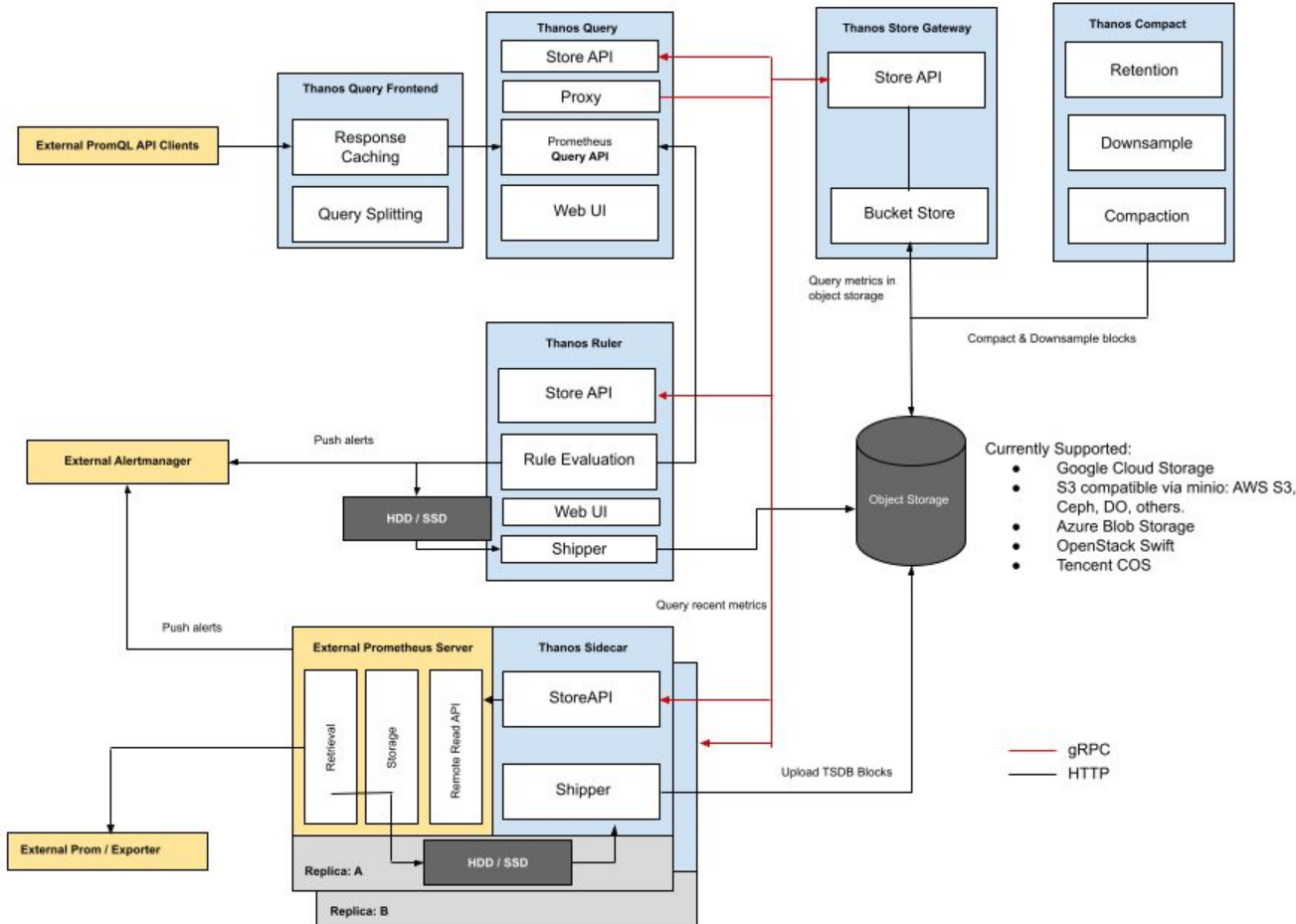
- The Prometheus additional setup for High Availability, Long Term metrics retention, and Global Query View. It is a CNCF-Incubated Project.
- Thanos consists of different components that packed into one binary or container image.
- Thanos can be deployed independently with only necessary components. Deployment can be flexible depending on our requirements.



<https://thanos.io>

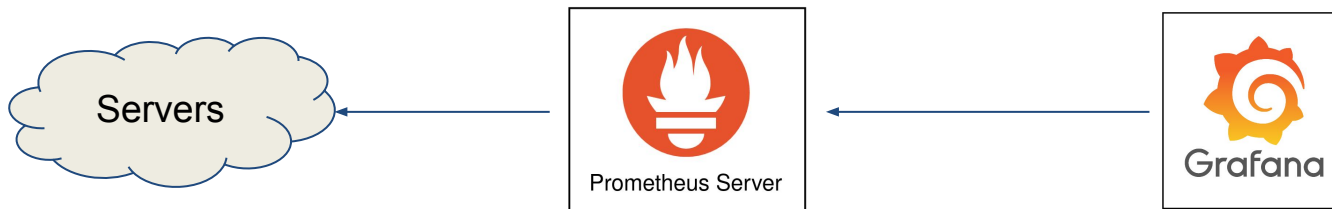
## Thanos Architecture

Source:  
<https://github.com/thanos-io/thanos>

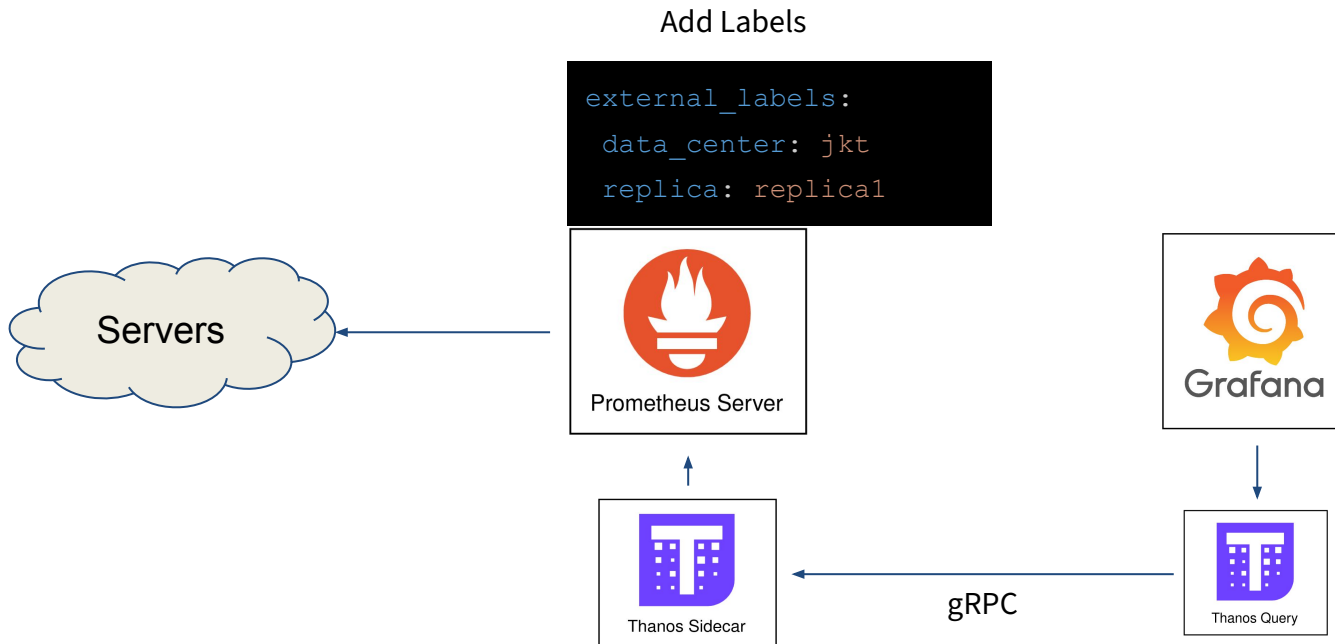


Confusing?  
Let's Break it down bottom-up!

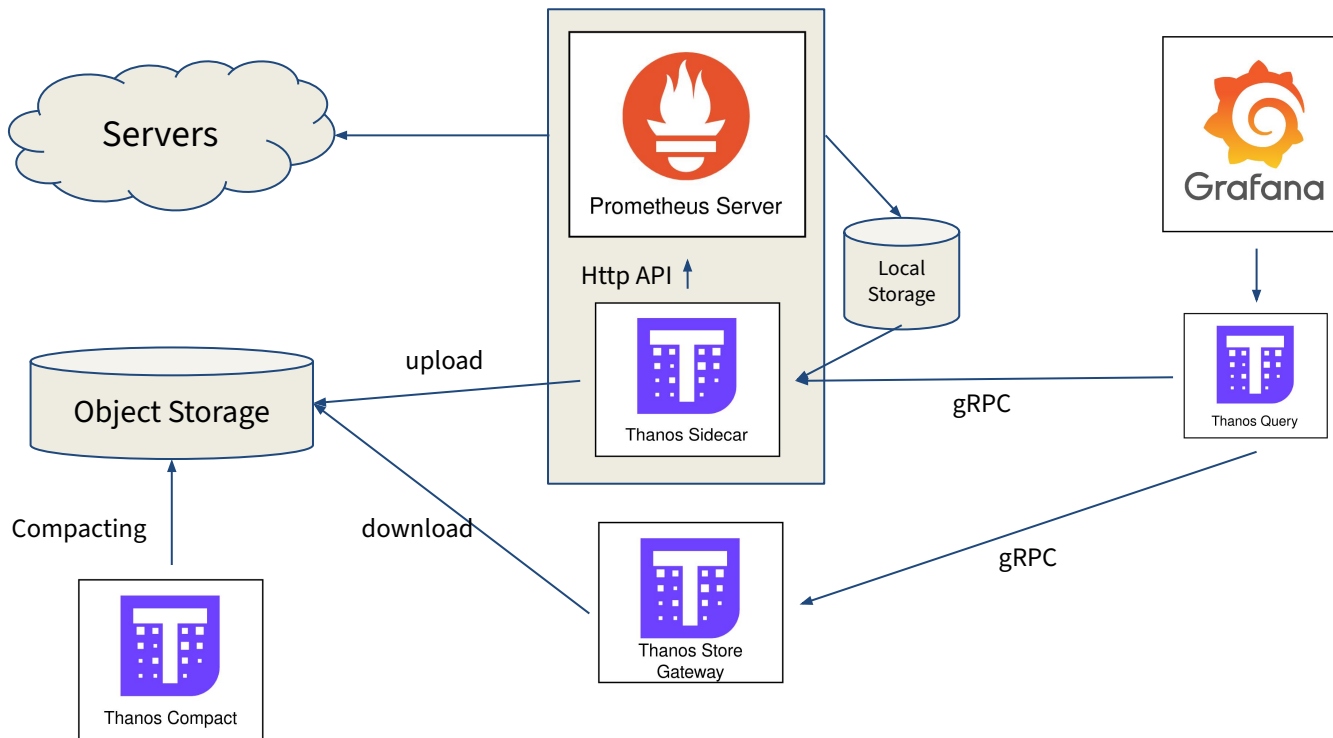
# Getting Started with Thanos

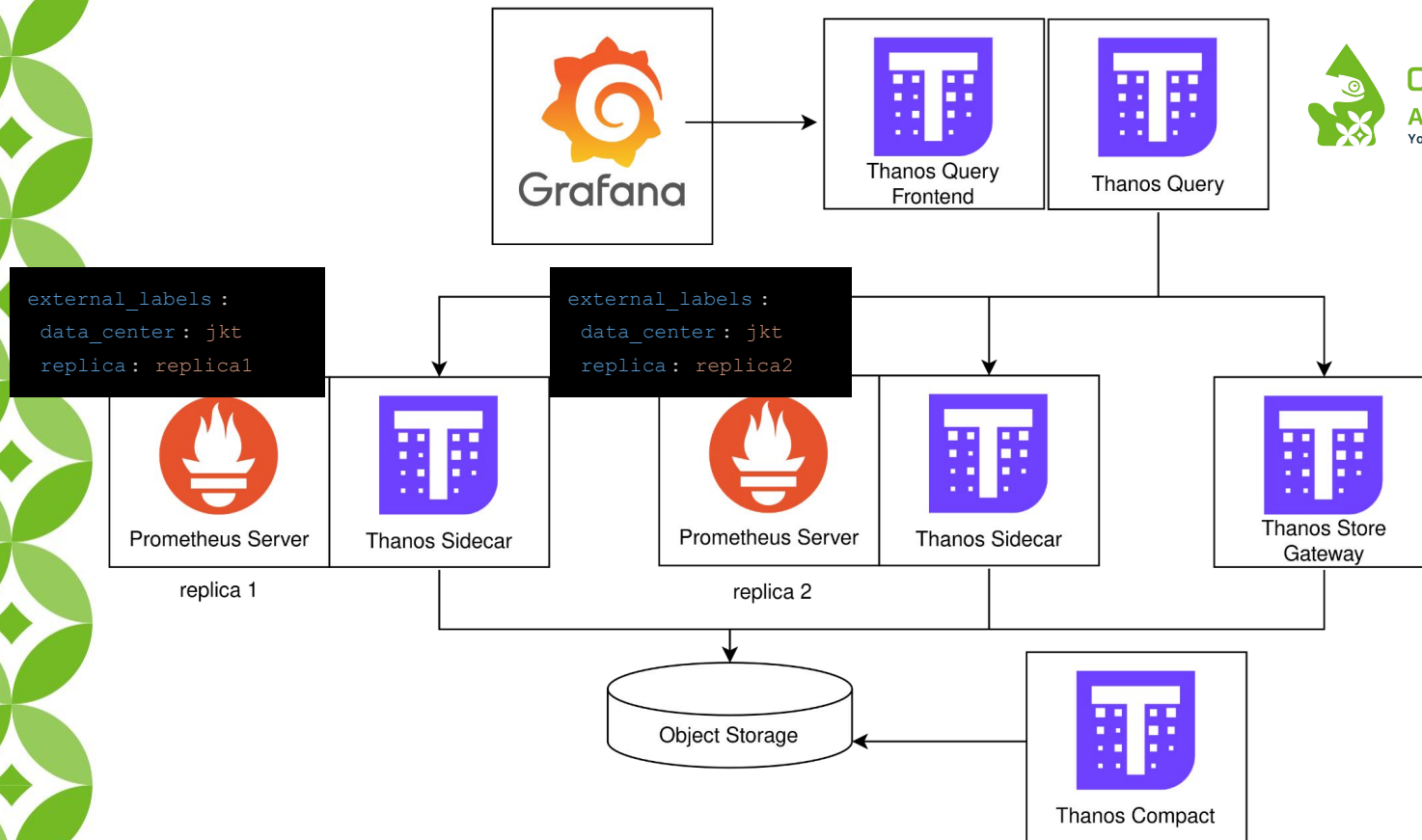


# Getting Started with Thanos

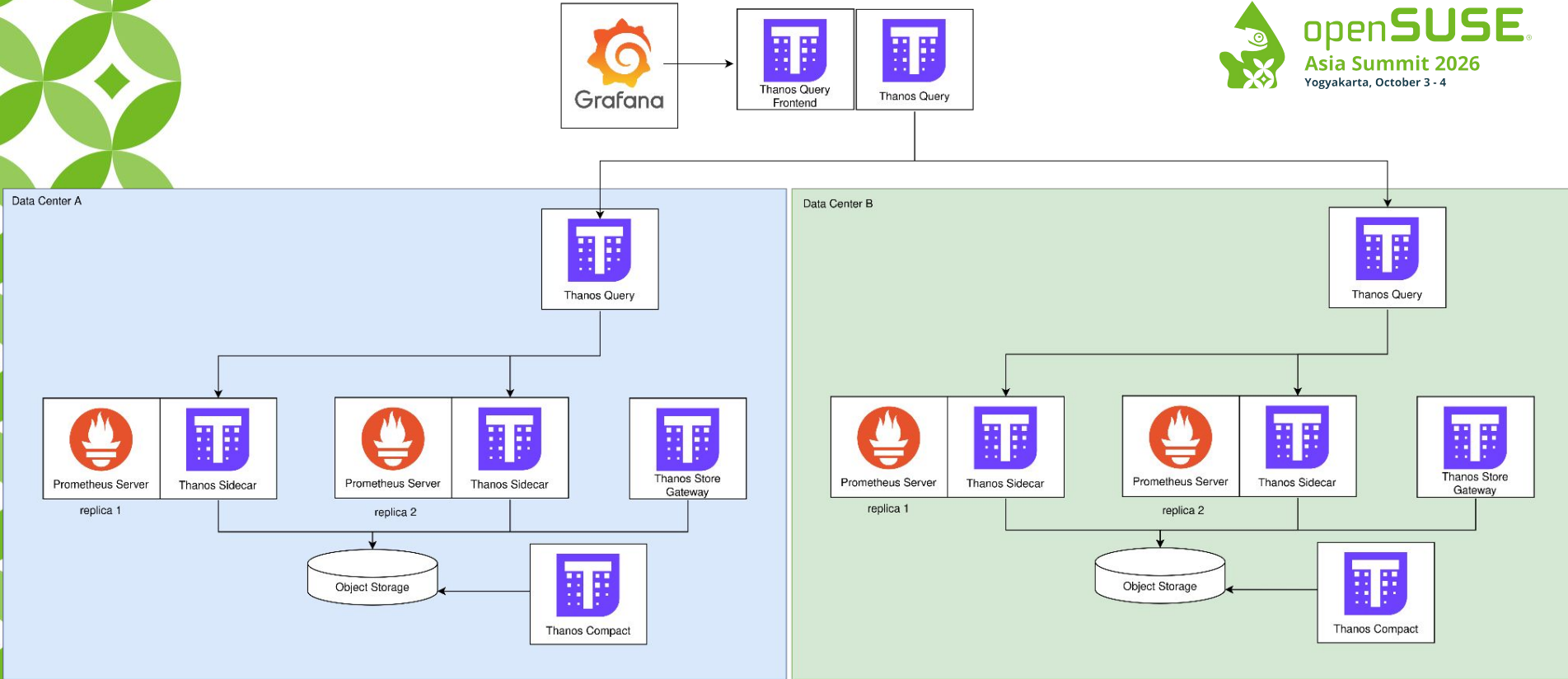


# Getting Started with Thanos





With added Thanos Sidecar and Thanos Store Gateway, we can achieve the High-Availability Prometheus with longer retention which is served by Thanos Store Gateway with Object Storage backing.



We can achieve the Global View Query with Long Term Data Retention and High Availability using this configuration. This is the typical starting point for upgrading our Monitoring Architecture to adapt with large number of servers.



**openSUSE®**  
Asia Summit 2026  
Yogyakarta, October 3 - 4

# Object Storage Retention with Thanos

# Object Storage Retention with Thanos

- Thanos can support many varieties of Object Storage. Be sure to check this URL for references: <https://thanos.io/tip/thanos/storage.md>
- The storing method is similar to Prometheus Data Storage which is using TSDB.
- Thanos can compress and downscale Metrics to 5m and 1h resolution for longer query requirements (be sure to adjust the PromQL query-range in your dashboard).
- Object Storage operation will be needed for Thanos Sidecar to upload the data block, Thanos Store Gateway to serve the data block from, and Thanos Compactor to compact the existing data blocks inside.

# TSDB Structure

```
gpmidprone2:~ # ll /opt/data/prometheus/
total 40
drwxr-xr-x. 1 gerald 2000    60 Sep  7 18:32 01M1YJ9KYRF6DTYWM7KMWZRBTG
drwxr-xr-x. 1 gerald 2000    60 Sep  7 19:00 01M1YKVS5KQ22S9Z604K03RXYJ
drwxr-xr-x. 1 gerald 2000    60 Sep  7 21:00 01M1YTQG8JHE7RTFK3TN0WDRKE
drwxr-xr-x. 1 gerald 2000    60 Sep  7 23:00 01M1Z1K7GK9GNE0MMPWQRGWH41
drwxr-xr-x. 1 gerald 2000    60 Sep  8 12:14 01M20F1FVR7DV5070NPCTYZMRW
drwxr-xr-x. 1 gerald 2000    60 Sep  8 15:14 01M20SB4FJZ7XC9WZGM5TQ5DJC
drwxr-xr-x. 1 gerald 2000    48 Sep  8 15:30 chunks_head
-rw-r--r--. 1 gerald 2000      0 Sep  8 15:03 lock
-rw-r--r--. 1 gerald 2000 20001 Sep  8 15:03 queries.active
drwxr-x---. 1 gerald 2000    12 Sep  7 18:33 thanos
-rw-r--r--. 1 gerald 2000   196 Sep  8 15:14 thanos.shipper.json
drwxr-xr-x. 1 gerald 2000   102 Sep  8 15:14 wal
gpmidprone2:~ #
```

It has the 0M1YXXXXX folder which represents the data block that has been saved by Prometheus. It will be uploaded into the Object Storage.

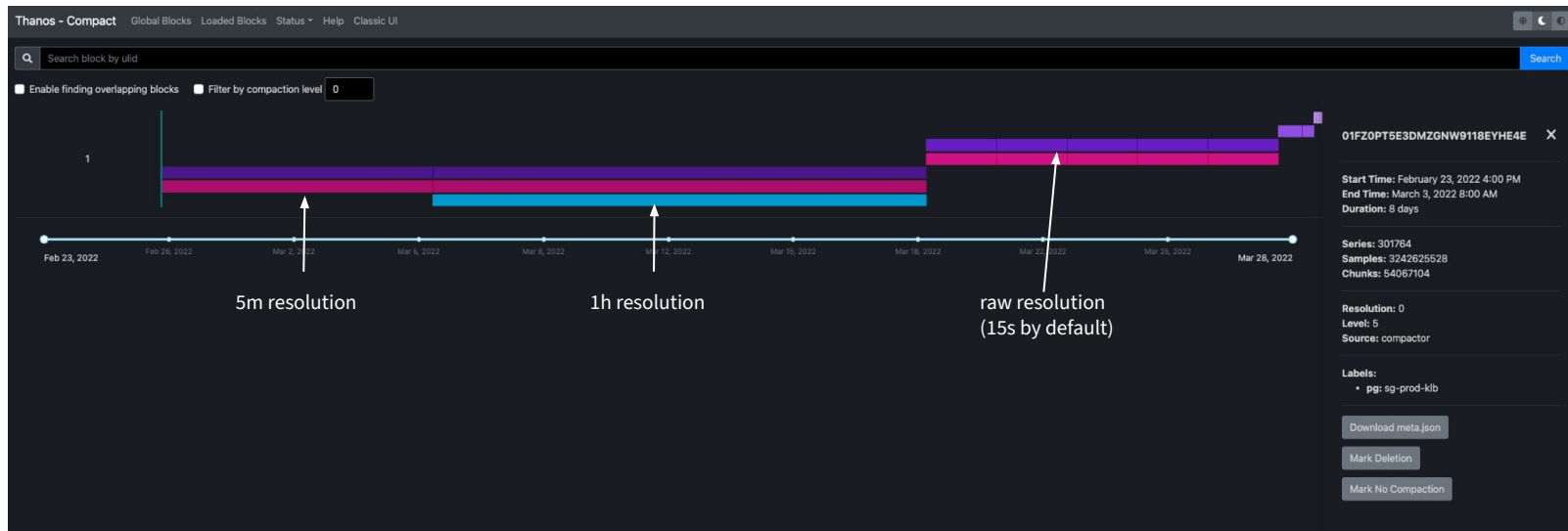
The thanos.shipper.json will keep track of which data blocks that has been uploaded.

Thanos Compactor will help to compact thos data blocks folder and creating new folder with compressed data results.

# Object Storage Downsampling

- Thanos Compactor component will help to compress freshly-uploaded data blocks in Object Storage to longer days and smaller sizes. It can also downsample the data blocks for longer query duration.
- Think of it as the reducing the resolution of time range with longer time data point (15s on default vs 5m or 1h resolution).
- 5m downsampling requires minimum data block age of 40 hours (around 2 days data block).
- 1h downsampling requires minimum data block age of 10 days (usually around 14 days data block).
- It is recommended to keep the raw data block and downsampled data together to enable detailed metric view at certain time range.

# TSDDB Visualization with Thanos



<https://www.sobyte.net/post/2022-03/thanos-guide/>

# Design Tips

# Design Tips with Thanos

- Separate Prometheus Instances for Node Exporter metrics with other Exporter metrics (Node Exporter metrics can be overwhelming).
- If you have servers spanning multiple Data Centers, be sure to deploy Prometheus for each Data Center if possible (data locality). Object storage can be centralized if you have no data locality problem.
- Thanos can be deployed with Podman or Kubernetes. Choose your most suitable deployment design.
- You might want to increase The `--storage.tsdb.min-block-duration` and `--storage.tsdb.max-block-duration` longer than 2h (x2 multiplier into 1d) to adapt on your existing Prometheus Instance (migration tips). It might be useful for querying hundreds of existing server metrics because the more data blocks are queried, the slower the query is.

# Example Design with Prometheus

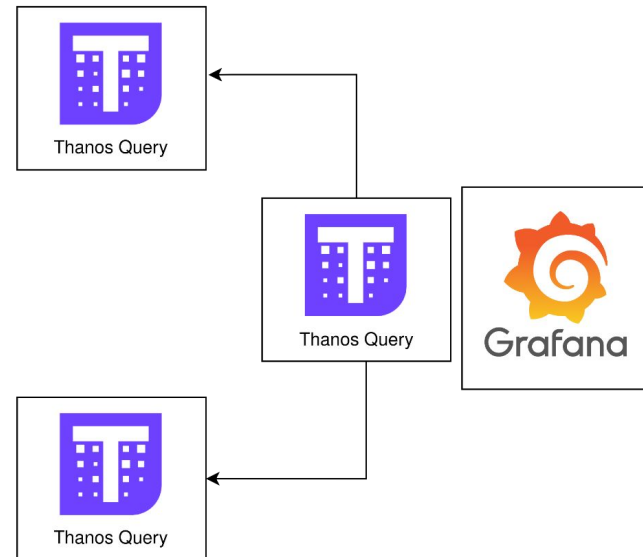
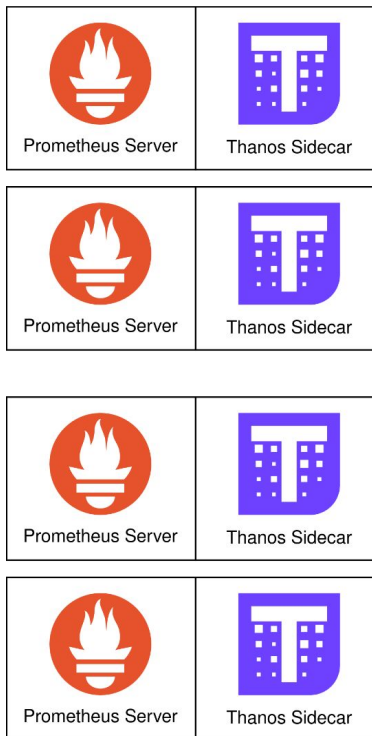
```
external_labels:  
  replica: replicaX  
  app: node_exporter
```

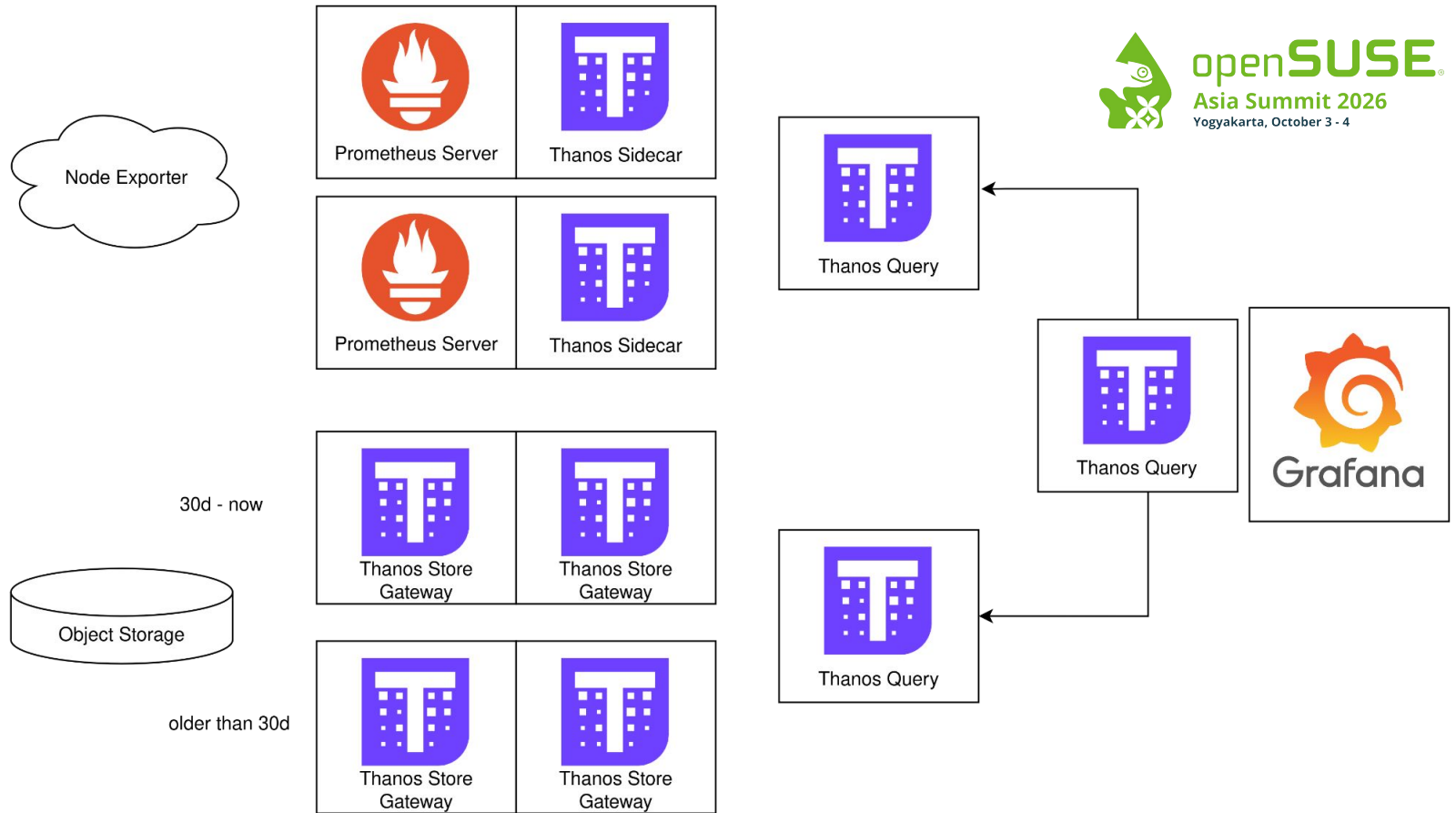
Node Exporter

```
external_labels:  
  replica: replicaX  
  app: general
```

cAdvisor Exporter

PostgreSQL Exporter





We can also replicate the Thanos Store Gateway Instances as High-Availability mode and providing some sharding by time range with `-min-time` and `-max-time`. It also can be sharded by hashmod (modulus of certain number of replicas).

# Demo Time!

<https://github.com/geraldapm/opensuse-prometheus-thanos>

## Demo Scenario

- The demo environment will be using KVM virtualization with my own laptop which is installed openSUSE LEAP 16.
- We will use a sort of script-assisted provisioning method with *ignition* for easy provisioning.
- The Prometheus, Grafana, and Thanos components will be deployed using Podman with *podman-kube* method. SSL configuration will be skipped for easy demonstration.
- The Object Storage will be using rustFS which is deployed outside of Demo VMs in the KVM host process.

## openSUSE Leap Micro

- An openSUSE OS distribution of lightweight OS with immutability and easy provisioning with ignition and combustion
- Available with pre-installed *podman*.
- Easy to provision-destroy with *ignition* with *butane* helper (yaml-based).
- Compatible with podman-based deployments or used for Kubernetes Cluster.

The Servers are deployed using openSUSE LEAP Micro because it has pre-installed Podman. Feel free to use another openSUSE variations because the concepts are similar.

## Demo Scenario

- Scenario 1: Typical Node Exporter - Prometheus - Grafana deployment.
- Scenario 2: Adding another prometheus replica with Thanos Sidecar and Thanos Query
- Scenario 3: Applying Object Storage Data Retention with rustFS, Thanos Store Gateway, and Thanos Compact
- Scenario 4: Adding another region with the same configuration as Scenario 3

## Demo Scenario

- The demo directory will take place in /opt because it is the directory that is writable on openSUSE Leap Micro.
- The /opt/config will be used to store Grafana, Prometheus, and Thanos configuration.
- It will also use the /opt/data to store the Prometheus data and Thanos Store and Thanos Compact necessary data directory to store data blocks.
- The /opt/deployment will be used to store the podman-kube yaml file, which is started by podman quadlet stored in /etc/containers/systemd.

# Installing Node Exporter

Node Exporter will be installed via ignition and triggered together with VM provisioning.

```
variant: fcos
version: 1.5.0
systemd:
  units:
    - name: install-node-exporter.service
      enabled: true
      contents: |
        [Unit]
        Description=Node Exporter Installer
        After=network-online.target
        Wants=network-online.target
        ConditionPathExists=!/opt/bin/node_exporter

        [Service]
        Type=oneshot
        RemainAfterExit=yes
        ExecStartPre=/usr/bin/mkdir -p /opt/local/bin
        ExecStart=/bin/sh -c '\
          TMP_DIR=/tmp && \
          curl -sSL "https://github.com/prometheus/node_exporter/releases/download/v1.12.1/node_exporter-1.12.1.linux-amd64.tar.gz" | tar -xz -C "$TMP_DIR" && \
          cp "$TMP_DIR/node_exporter-1.12.1.linux-amd64/node_exporter" /opt/local/bin/ && \
          chmod +x /opt/local/bin/node_exporter && \
          rm -rf "$TMP_DIR/node_exporter-1.12.1.linux-amd64"'

        [Install]
        WantedBy=multi-user.target
```

# Installing Node Exporter

Then it will be started by `node_exporter.service` to enable systemd autostart.

```
- name: node_exporter.service
  enabled: true
  contents: |
    [Unit]
    Description=Node Exporter Metrics Agent
    After=network-online.target
    Wants=network-online.target

    [Service]
    Restart=always
    RestartSec=5s
    EnvironmentFile=--/etc/default/%N
    EnvironmentFile=--/etc/sysconfig/%N
    EnvironmentFile=--/etc/systemd/system/node_exporter
    ExecStartPre=mkdir -p /var/lib/node_exporter/textfile_collector
    ExecStart=/opt/local/bin/node_exporter \
      --collector.textfile.directory /var/lib/node_exporter/textfile_collector \
      --no-collector.wifi \
      --no-collector.hwmon \
      --no-collector.btrfs \
      --collector.filesystem.mount-points-exclude='^(dev|proc|sys|run|k3s/containerd|.+|var/lib/docker|.+|var/lib/kubelet/pods|.+)($/)' \
      --collector.netclass.ignored-devices='^(veth.*|[a-f0-9]{15})$' \
      --collector.netdev.device-exclude='^(veth.*|[a-f0-9]{15})$'

    [Install]
    WantedBy=multi-user.target
```

# Podman Quadlet

We will use podman-kube and podman quadlet to run our Grafana, Prometheus, and Thanos containers. It is a way to simulate kubernetes pod on podman because we only need to utilize podman to run our containers. It also enables us to provide autostart for the containers.

We can also use systemd to restart the containers by running `systemctl restart grafana`

`/etc/containers/systemd/grafana.kube`

```
[Unit]
Description=Grafana Deployment

[Kube]
Yaml=/opt/deployment/grafana.yaml

[Install]
# Start by default on boot
WantedBy=multi-user.target default.target
```

`/etc/containers/systemd/prometheus.kube`

```
[Unit]
Description=Prometheus Deployment

[Kube]
Yaml=/opt/deployment/prometheus.yaml

[Install]
# Start by default on boot
WantedBy=multi-user.target default.target
```

# Podman Kube

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: grafana
    name: grafana
spec:
  containers:
    - name: grafana
      image: docker.io/grafana/grafana-oss
      ports:
        - containerPort: 3000
          hostPort: 3000
      resources: {}
      volumeMounts:
        - mountPath: /etc/grafana/provisioning
          name: grafana_provisioning
        - mountPath: /var/lib/grafana/dashboards
          name: grafana_dashboards
      args:
        - /bin/bash
        - -c
      restartPolicy: OnFailure
  volumes:
    - name: grafana_provisioning
      hostPath:
        path: /opt/config/grafana/provisioning
        type: Directory
    - name: grafana_dashboards
      hostPath:
        path: /opt/config/grafana/dashboards
        type: Directory
  status: {}
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    app: prometheus
    name: prometheus
spec:
  containers:
    - name: prometheus
      image: docker.io/prom/prometheus:latest
      ports:
        - containerPort: 9090
          hostPort: 9090
      resources: {}
      runAsUser: 1000
      runAsGroup: 2000
      fsGroup: 2000
      runAsNonRoot: true
      volumeMounts:
        - mountPath: /prometheus/config
          name: prometheus_config
          readOnly: true
        - mountPath: /prometheus/data
          name: prometheus_data
      args:
        - "--config.file=/prometheus/config/prometheus.yml"
        - "--web.enable-remote-write-receiver"
        - "--web.enable-lifecycle"
      restartPolicy: OnFailure
  volumes:
    - name: prometheus_config
      hostPath:
        path: /opt/config/prometheus
        type: File
    - name: prometheus_data
      hostPath:
        path: /opt/data/prometheus
        type: Directory
  status: {}
```

# Installing Grafana

It will be using simple ssh and scp to copy the config file and podman-kube file to each corresponding server. We can use bash scripting to achieve this.

```
#!/bin/bash
GRAFANA_SERVER=192.168.100.30

# Deploy Grafana Configs
ssh root@$GRAFANA_SERVER mkdir -p /etc/containers/systemd
ssh root@$GRAFANA_SERVER mkdir -p /opt/{config,deployment,data}/grafana
scp systemd/grafana.kube root@$GRAFANA_SERVER:/etc/containers/systemd/grafana.kube
scp deployment/grafana.yaml root@$GRAFANA_SERVER:/opt/deployment/grafana.yaml
scp -r config/grafana root@$GRAFANA_SERVER:/opt/config/
ssh root@$GRAFANA_SERVER systemctl daemon-reload
ssh root@$GRAFANA_SERVER systemctl restart grafana
```

# Installing Prometheus

We also utilized ssh and scp to copy all the necessary configs. In this case, the prometheus data folder will need to have all 777 permission, specific ownership, and the SELinux context.

```
#!/bin/bash
PROMETHEUS_SERVER=192.168.100.10

# Deploy Prometheus Configs
ssh root@$PROMETHEUS_SERVER mkdir -p /etc/containers/systemd
ssh root@$PROMETHEUS_SERVER mkdir -p /opt/{config,deployment,data}/prometheus
scp systemd/prometheus.kube root@$PROMETHEUS_SERVER:/etc/containers/systemd/prometheus.kube
scp deployment/prometheus.yaml root@$PROMETHEUS_SERVER:/opt/deployment/prometheus.yaml
scp -r config/prometheus root@$PROMETHEUS_SERVER:/opt/config/
ssh root@$PROMETHEUS_SERVER "mkdir -p /opt/data/prometheus && chmod 777 /opt/data/prometheus"
ssh root@$PROMETHEUS_SERVER "chown -R 1000:2000 /opt/data/prometheus/*"
ssh root@$PROMETHEUS_SERVER 'semanage fcontext -at container_file_t
"/opt/data/prometheus(/.*)?"; restorecon -Rv /opt/data/prometheus'
ssh root@$PROMETHEUS_SERVER systemctl daemon-reload
ssh root@$PROMETHEUS_SERVER systemctl restart prometheus
```

# Installing Prometheus

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    data_center: id
    replica: prome1

scrape_configs:
  - job_name: "node_exporter"
    static_configs:
      - targets:
        - gpmidservera:9100
        - gpmidserverb:9100
        - gpmidpromel:9100
        - gpmidgrafana:9100
```

- The prometheus.yml config is configured only to scrape the node exporter metrics from our defined servers.
- We defined the IP Address-Hostname mapping in /etc/hosts for each of prometheus servers. It is populated when we provision the VMs using ignition config.
- The external labels was defined in the beginning to ensure all of our metrics are labeled.

# Installing Prometheus

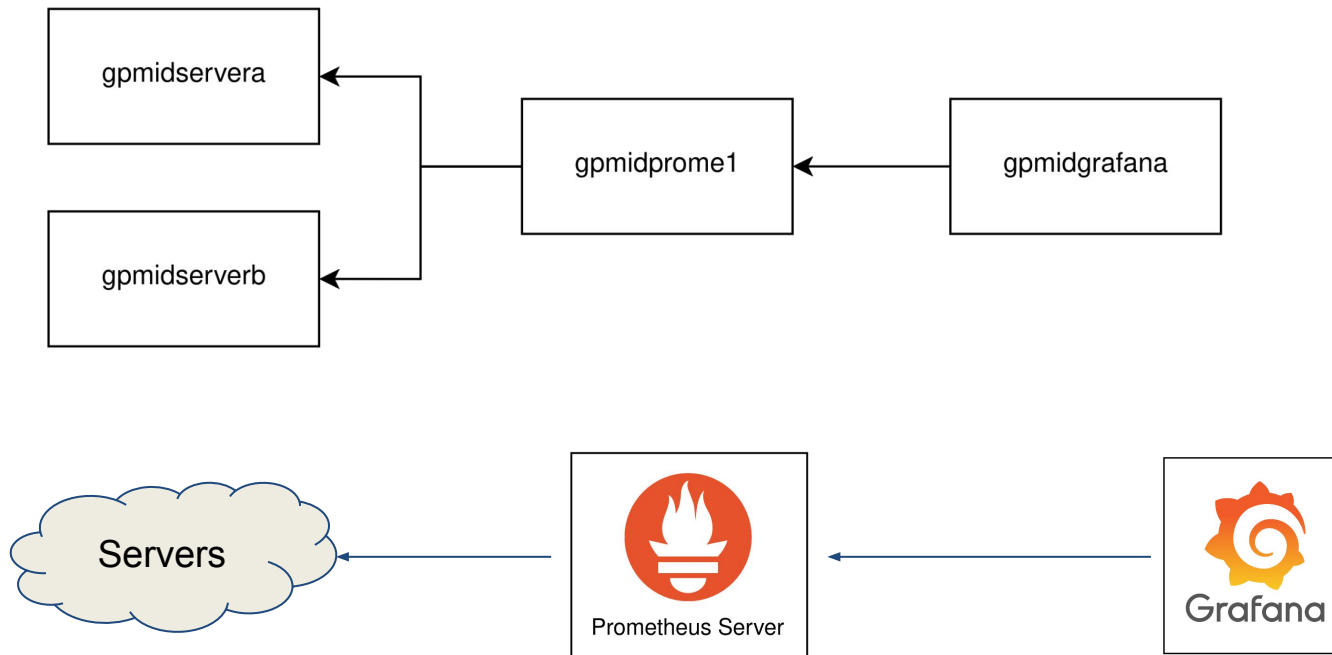
```
scrape_configs:
  - job_name: "node_exporter"
    relabel_configs:
      - source_labels: [ __address__ ]
        separator: ':'
        regex: '(.*) :9100'
        replacement: '${1}'
        target_label: instance
    static_configs:
      - targets:
          - gpmidservera:9100
          - gpmidserverb:9100
          - gpmidpromereplica:9100
          - gpmidgrafana:9100
```

- You might want to add relabeling config to display only the hostname and stripe-out the port number.
- Use the relabeling config to add the necessary relabeling.
- It will be useful when we combine it with the similar node exporter configuration from Thanos Query.
- You can get the differences when doing the demo for **Scenario 2** to **Scenario 4**.

# Prereq: Deploy the Node Exporter Client VMs

- Deploy the VMs
  - Go to the repository folder of `deployment/client`
  - Generate the ignition script: `bash generate-ignition.sh`
  - Start the VMs: `bash start-vm.sh --provision`
  - Stop the VMs if unused: `bash stop-vm.sh`

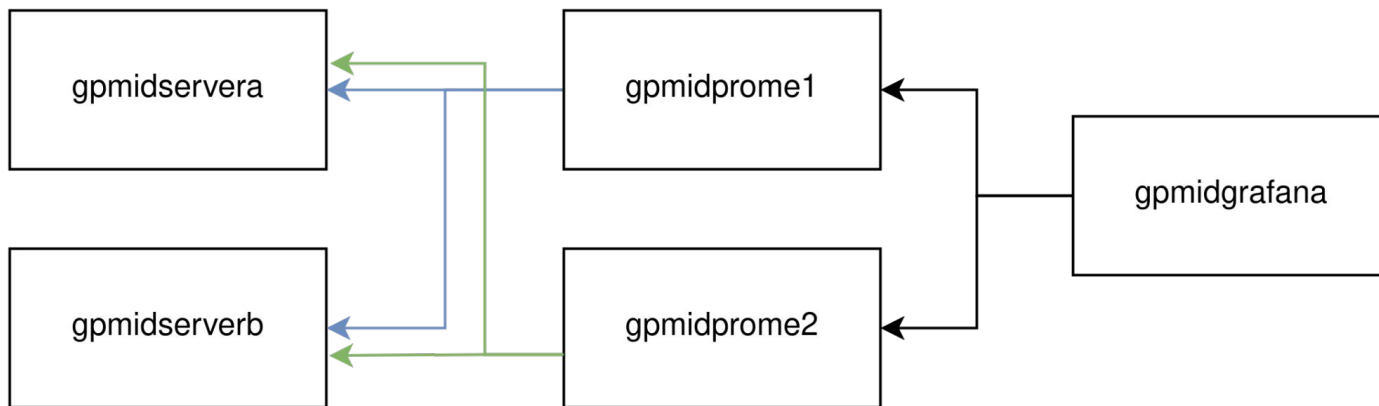
# Scenario 1: Typical Node Exporter - Prometheus - Grafana deployment



# Scenario 1: Typical Node Exporter - Prometheus - Grafana deployment

- Deploy the VMs
  - Go to the repository folder of `deployment/scenario1`
  - Generate the ignition script: `bash generate-ignition.sh`
  - Start the VMs: `bash start-vm.sh --provision`
  - Stop the VMs if unused: `bash stop-vm.sh`
- Deploy the Containers
  - Go to the repository folder of `podman/scenario1`
  - Run the script to deploy the config: `bash deploy.sh`
  - If you want to clean the environment, run: `bash cleanup.sh`

## Scenario 2: Adding another prometheus replica with Thanos Sidecar and Thanos Query



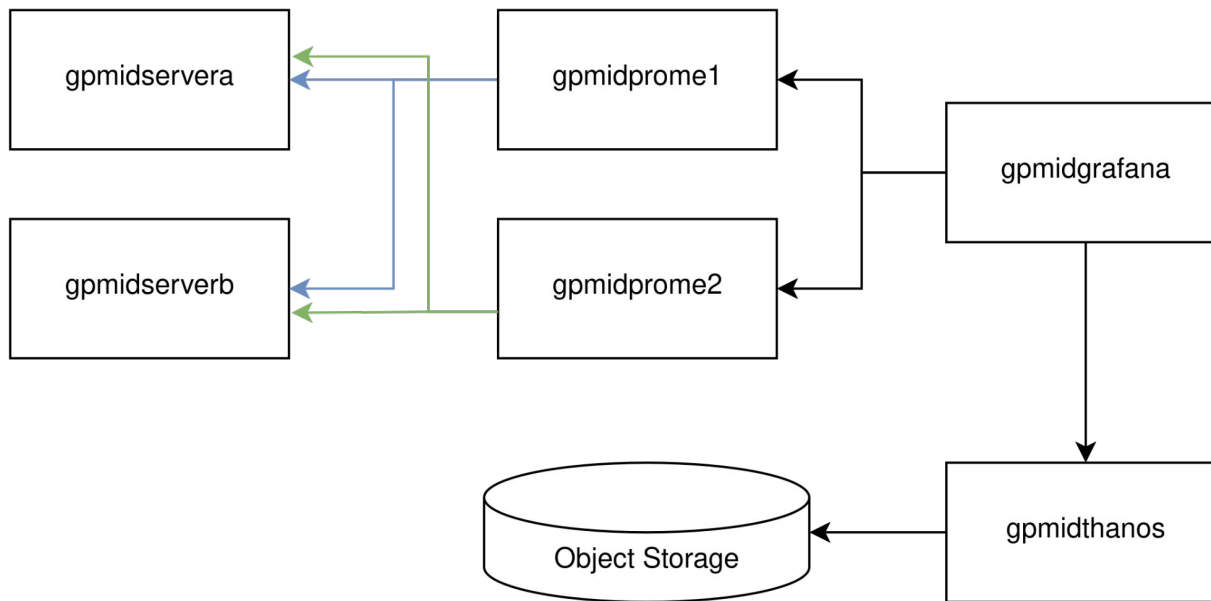
## Scenario 2: Adding another prometheus replica with Thanos Sidecar and Thanos Query

- Deploy the VMs
  - Go to the repository folder of `deployment/scenario2`
  - Generate the ignition script: `bash generate-ignition.sh`
  - Start the VMs: `bash start-vm.sh --provision`
  - Stop the VMs if unused: `bash stop-vm.sh`
- Deploy the Containers
  - Go to the repository folder of `podman/scenario2`
  - Run the script to deploy the config: `bash deploy.sh`
  - If you want to clean the environment, run: `bash cleanup.sh`

## Ensure that Prometheus and Thanos Sidecar running with the same UID and GID to prevent this error:

```
ts=2026-09-07T22:58:31.559921358Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T22:58:31.560663057Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
ts=2026-09-07T22:59:01.55941784Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T22:59:01.561592109Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
ts=2026-09-07T22:59:31.559552543Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T22:59:31.560407665Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
ts=2026-09-07T23:00:01.558705931Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T23:00:01.559594504Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
ts=2026-09-07T23:00:31.558118105Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T23:00:31.559032805Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
ts=2026-09-07T23:01:01.559456351Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T23:01:01.560342758Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
ts=2026-09-07T23:01:31.558355589Z caller=shipper.go:479 level=info msg="upload new block" id=01M1YJ4ARQ7X8QBAYJKN2685X9
ts=2026-09-07T23:01:31.559352554Z caller=sidecar.go:483 level=warn err="upload 01M1YJ4ARQ7X8QBAYJKN2685X9: hard link block: hard link file chunks/000001: link
/prometheus/data/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001 /prometheus/data/thanos/upload/01M1YJ4ARQ7X8QBAYJKN2685X9/chunks/000001: operation not permitted" uploaded=0
```

## Scenario 3: Applying Object Storage Data Retention with rustFS (Object Storage), Thanos Store Gateway, and Thanos Compact

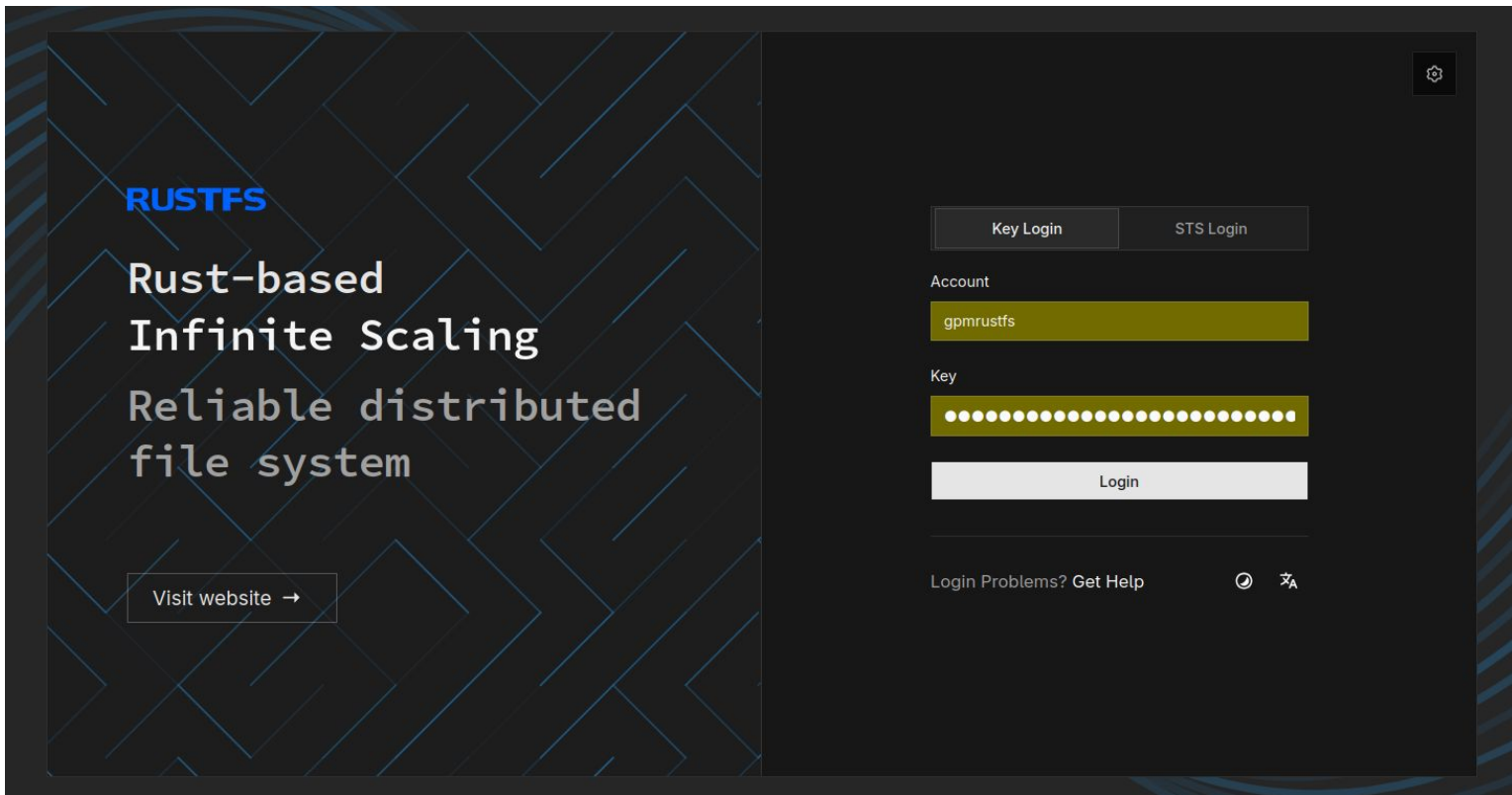


# Scenario 3: Applying Object Storage Data Retention with rustFS (Object Storage), Thanos Store Gateway, and Thanos Compact

- Deploy the VMs
  - Go to the repository folder of `deployment/scenario3`
  - Generate the ignition script: `bash generate-ignition.sh`
  - Start the VMs: `bash start-vm.sh --provision`
  - Stop the VMs if unused: `bash stop-vm.sh`
- Deploy the Containers
  - Go to the repository folder of `podman/scenario3`
  - Start the rustFS Object Storage by running: `bash deploy-rustfs.sh`
  - Run the script to deploy the config: `bash deploy.sh`
  - If you want to clean the environment, run: `bash cleanup.sh`

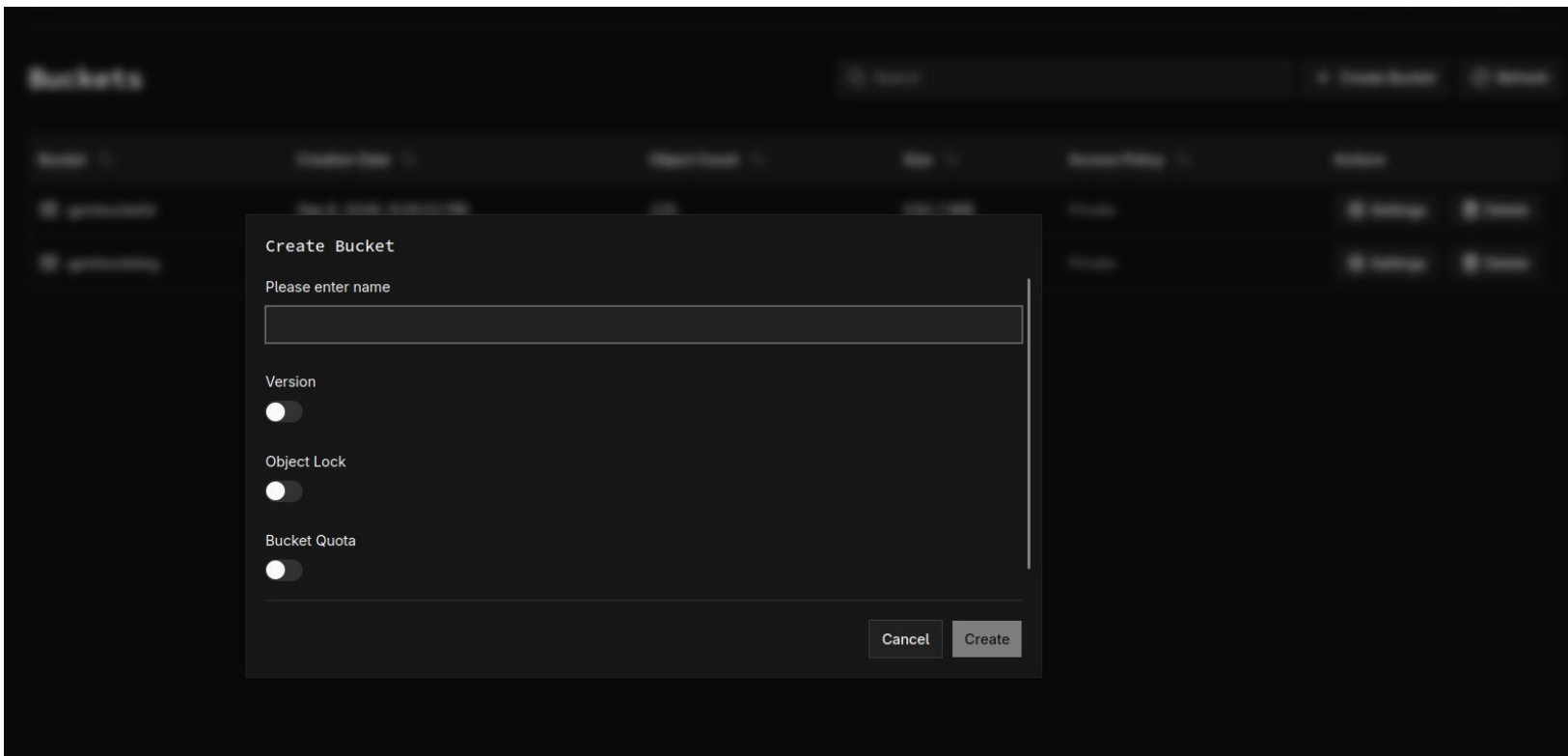
# Setup rustfs for Object Storage Provider

Access `localhost:9001`

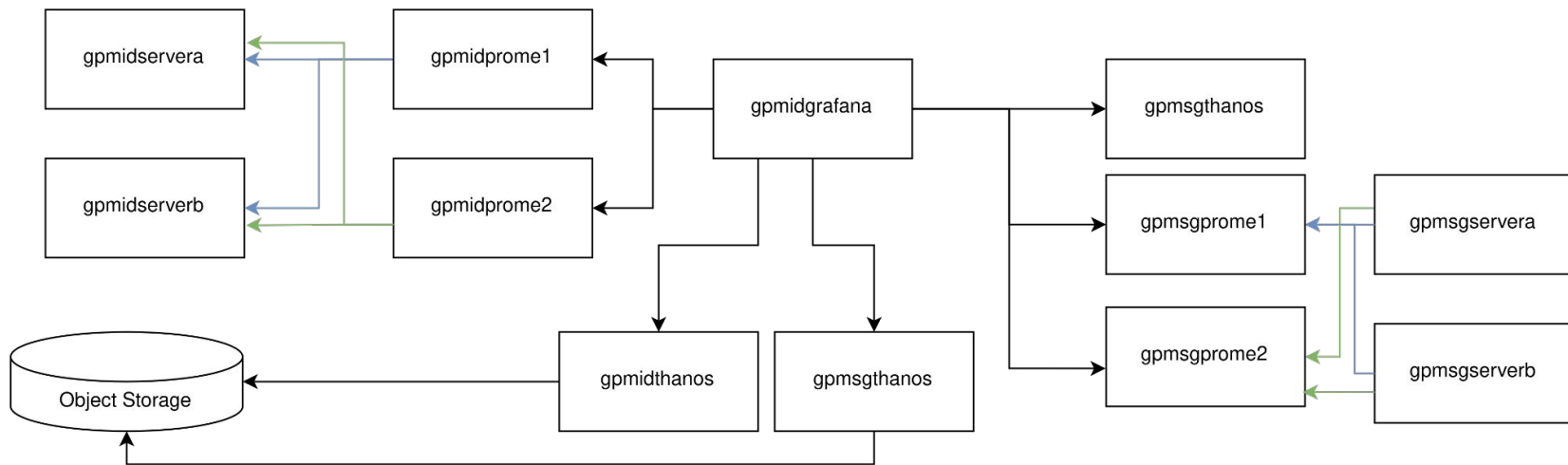


# Setup rustfs for Object Storage Provider

Click on “Create Bucket” on upper right corner



## Scenario 4: Adding another region with the same configuration as Scenario 3



## Scenario 4: Adding another region with the same configuration as Scenario 3

- Deploy the VMs
  - Go to the repository folder of `deployment/scenario4`
  - Generate the ignition script: `bash generate-ignition.sh`
  - Start the VMs: `bash start-vm.sh --provision`
  - Stop the VMs if unused: `bash stop-vm.sh`
- Deploy the Containers
  - Go to the repository folder of `podman/scenario4`
  - Start the rustFS Object Storage by running: `bash deploy-rustfs.sh`
  - Run the script to deploy the config: `bash deploy.sh`
  - If you want to clean the environment, run: `bash cleanup.sh`

# Demo Results

Grafana: <https://grafana-osas26.gpm.my.id>

Thanos Query: <https://thanos-query-osas26.gpm.my.id>

Purposefully tunneled from Cloudflare to my own laptop for better visualization :D

# Final Result



Thanos - Query

Graph

Endpoints

Status ▾

Help



## Sidcar

[show less](#)

| Endpoint          | Status | Announced LabelSets                                         | Min Time (UTC)      | Max Time (UTC) | Last Successful Health Check | Last Message |
|-------------------|--------|-------------------------------------------------------------|---------------------|----------------|------------------------------|--------------|
| gpmidprome1:10900 | UP     | <code>data_center="id"</code> <code>replica="prome1"</code> | 2026-09-08 12:14:16 | —              | 4.928s ago                   |              |
| gpmidprome2:10900 | UP     | <code>data_center="id"</code> <code>replica="prome2"</code> | 2026-09-08 12:14:16 | —              | 4.928s ago                   |              |
| gpmsgprome1:10900 | UP     | <code>data_center="sg"</code> <code>replica="prome1"</code> | 2026-09-08 14:40:35 | —              | 4.948s ago                   |              |
| gpmsgprome2:10900 | UP     | <code>data_center="sg"</code> <code>replica="prome2"</code> | 2026-09-08 14:41:33 | —              | 4.950s ago                   |              |

## Store

[show less](#)

| Endpoint          | Status | Announced LabelSets                                         | Min Time (UTC)      | Max Time (UTC)      | Last Successful Health Check | Last Message |
|-------------------|--------|-------------------------------------------------------------|---------------------|---------------------|------------------------------|--------------|
| gpmidthanos:10902 | UP     | <code>data_center="id"</code>                               | 2026-09-08 12:14:16 | 2026-09-09 06:00:00 | 4.858s ago                   |              |
|                   |        | <code>data_center="id"</code> <code>replica="prome1"</code> |                     |                     |                              |              |
|                   |        | <code>data_center="id"</code> <code>replica="prome2"</code> |                     |                     |                              |              |
| gpmsgthanos:10902 | UP     | <code>data_center="sg"</code>                               | 2026-09-08 14:40:35 | 2026-09-09 06:00:00 | 4.931s ago                   |              |
|                   |        | <code>data_center="sg"</code> <code>replica="prome1"</code> |                     |                     |                              |              |
|                   |        | <code>data_center="sg"</code> <code>replica="prome2"</code> |                     |                     |                              |              |

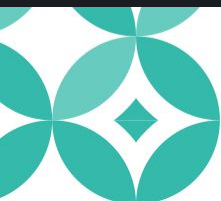
# Targets

All Unhealthy Collapse All

Filter by endpoint or labels

node\_exporter (11/12 up) [show less](#)

| Endpoint                                                                          | State | Labels                                                                                                    | Last Scrape | Scrape Duration | Error                                                                                            |
|-----------------------------------------------------------------------------------|-------|-----------------------------------------------------------------------------------------------------------|-------------|-----------------|--------------------------------------------------------------------------------------------------|
| <a href="http://gpmidgrafana:9100/metrics">http://gpmidgrafana:9100/metrics</a>   | UP    | <code>data_center="id"</code> <code>instance="gpmidgrafana:9100"</code> <code>job="node_exporter"</code>  | 3.240s      | 12.877ms        |                                                                                                  |
| <a href="http://gpmidprom1:9100/metrics">http://gpmidprom1:9100/metrics</a>       | UP    | <code>data_center="id"</code> <code>instance="gpmidprom1:9100"</code> <code>job="node_exporter"</code>    | 3.160s      | 15.503ms        |                                                                                                  |
| <a href="http://gpmidprom2:9100/metrics">http://gpmidprom2:9100/metrics</a>       | UP    | <code>data_center="id"</code> <code>instance="gpmidprom2:9100"</code> <code>job="node_exporter"</code>    | 553.000ms   | 18.378ms        |                                                                                                  |
| <a href="http://gpmidservera:9100/metrics">http://gpmidservera:9100/metrics</a>   | UP    | <code>data_center="id"</code> <code>instance="gpmidservera:9100"</code> <code>job="node_exporter"</code>  | 11.431s     | 28.005ms        |                                                                                                  |
| <a href="http://gpmidserverb:9100/metrics">http://gpmidserverb:9100/metrics</a>   | UP    | <code>data_center="id"</code> <code>instance="gpmidserverb:9100"</code> <code>job="node_exporter"</code>  | 3.955s      | 19.292ms        |                                                                                                  |
| <a href="http://gpmidthanos:9100/metrics">http://gpmidthanos:9100/metrics</a>     | UP    | <code>data_center="id"</code> <code>instance="gpmidthanos:9100"</code> <code>job="node_exporter"</code>   | 9.557s      | 13.951ms        |                                                                                                  |
| <a href="http://gpmmsggrafana:9100/metrics">http://gpmmsggrafana:9100/metrics</a> | DOWN  | <code>data_center="sg"</code> <code>instance="gpmmsggrafana:9100"</code> <code>job="node_exporter"</code> | 8.589s      | 3.60s           | Get "http://gpmmsggrafana:9100/metrics": dial tcp 192.168.101.30:9100: connect: no route to host |
| <a href="http://gpmmsgprom1:9100/metrics">http://gpmmsgprom1:9100/metrics</a>     | UP    | <code>data_center="sg"</code> <code>instance="gpmmsgprom1:9100"</code> <code>job="node_exporter"</code>   | 1.776s      | 32.493ms        |                                                                                                  |
| <a href="http://gpmmsgprom2:9100/metrics">http://gpmmsgprom2:9100/metrics</a>     | UP    | <code>data_center="sg"</code> <code>instance="gpmmsgprom2:9100"</code> <code>job="node_exporter"</code>   | 10.627s     | 9.574ms         |                                                                                                  |
| <a href="http://gpmmsgservera:9100/metrics">http://gpmmsgservera:9100/metrics</a> | UP    | <code>data_center="sg"</code> <code>instance="gpmmsgservera:9100"</code> <code>job="node_exporter"</code> | 442.000ms   | 10.741ms        |                                                                                                  |
| <a href="http://gpmmsgserverb:9100/metrics">http://gpmmsgserverb:9100/metrics</a> | UP    | <code>data_center="sg"</code> <code>instance="gpmmsgserverb:9100"</code> <code>job="node_exporter"</code> | 1.245s      | 30.946ms        |                                                                                                  |
| <a href="http://gpmmsgthanos:9100/metrics">http://gpmmsgthanos:9100/metrics</a>   | UP    | <code>data_center="sg"</code> <code>instance="gpmmsgthanos:9100"</code> <code>job="node_exporter"</code>  | 4.749s      | 15.525ms        |                                                                                                  |



The prometheus targets are aggregated in single-point viewing using Thanos Query. If you have defined rules on each prometheus, it will also be aggregated here.

☐ Use local time

☐ Enable query history

☐ Enable Store Filtering

☒ Enable autocomplete

☒ Enable highlighting

☒ Enable linter

Q

up{instance="gpmidgrafana:9100"}

Execute

Explain

☒ Use Deduplication

☒ Use Partial Response

☐ Force Tracing

Engine

Prometheus

▼

Analyze

Load time: 41ms

Resolution: 14s

Result series: 1

Table

Graph

<

Evaluation time

>

up{data\_center="id", instance="gpmidgrafana:9100", job="node\_exporter"}

1

Remove Panel

With deduplication

☐ Use local time

☐ Enable query history

☐ Enable Store Filtering

☒ Enable autocomplete

☒ Enable highlighting

☒ Enable linter

Q

up{instance="gpmidgrafana:9100"}

Execute

Explain

☐ Use Deduplication

☒ Use Partial Response

☐ Force Tracing

Engine

Prometheus

▼

Analyze

Load time: 20ms

Resolution: 14s

Result series: 2

Table

Graph

<

Evaluation time

>

up{data\_center="id", instance="gpmidgrafana:9100", job="node\_exporter", replica="prom1"}

1

up{data\_center="id", instance="gpmidgrafana:9100", job="node\_exporter", replica="prom2"}

1

Without deduplication

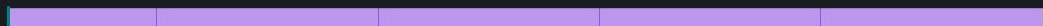
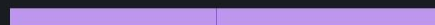
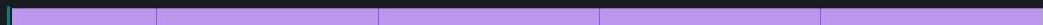
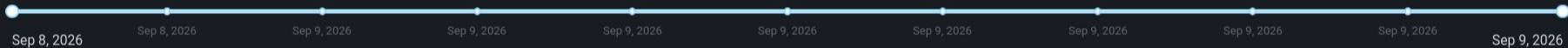
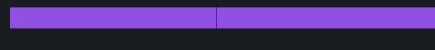


Search block by ulid

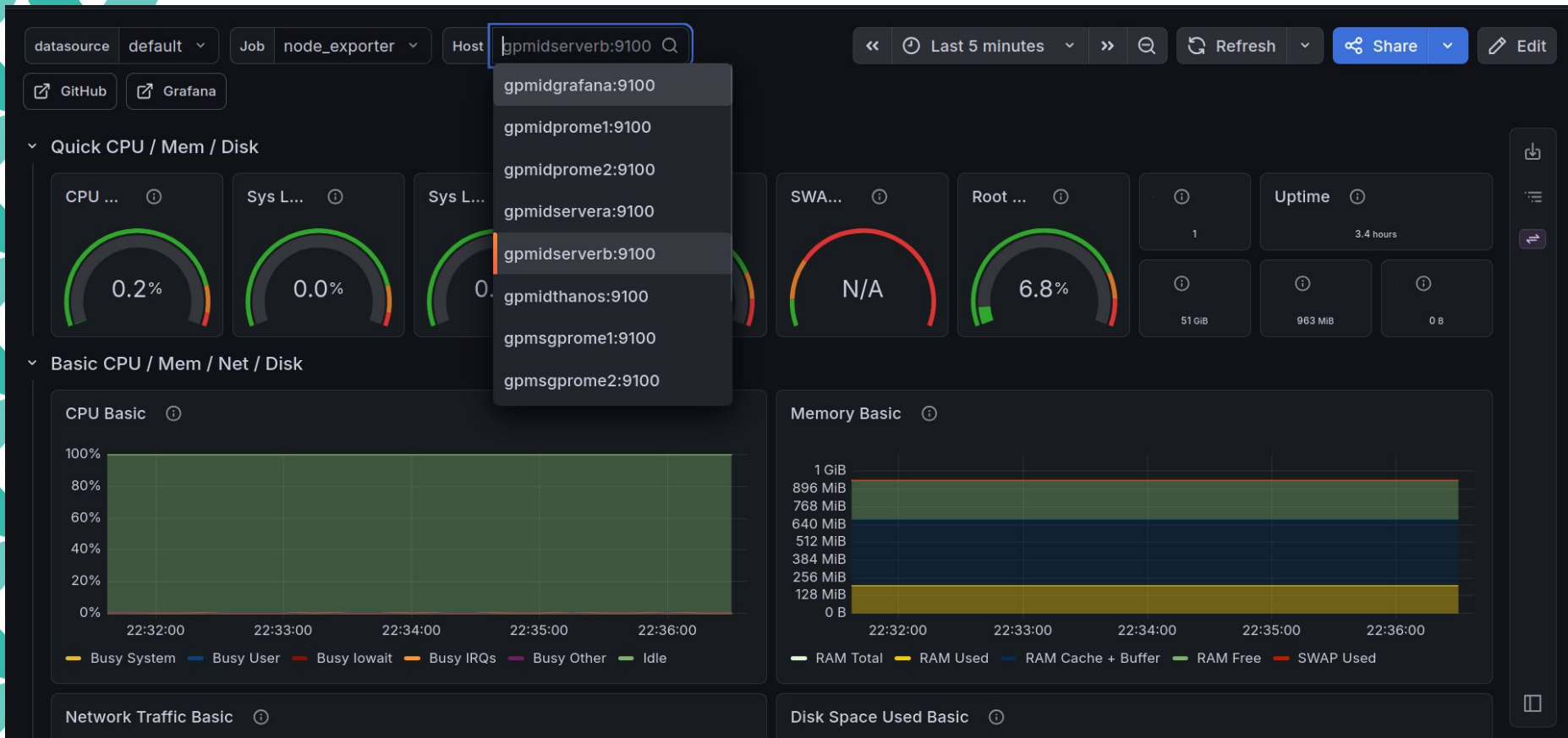
Search

☐ Enable finding overlapping blocks☐ Filter by compaction level

0

1  
14.70 MiB2  
14.67 MiB3  
66.66 MiB

Thanos Store Gateway served data blocks. The third row represents vertical-compacted data blocks.



Grafana will still view as one datasource with aggregated targets and metric query endpoint

# Extra Demo!

How about deploying it into a Kubernetes Cluster?  
Let's find out!

# Extra Scenario: Deploy on Kubernetes

- Deploy the Kubernetes Cluster of your choice
  - Go to the github repository of  
<https://github.com/geraldapm/microos-rke2> or  
<https://github.com/geraldapm/microos-k3s> or  
<https://github.com/geraldapm/microos-kubeadm>
  - Follow the Guide for deploying the cluster
  - You should have this configuration by the end of tutorial:

```
gpmrke2controlplane1:~ # kubectl get node
```

| NAME                 | STATUS | ROLES              | AGE | VERSION        |
|----------------------|--------|--------------------|-----|----------------|
| gpmrke2controlplane1 | Ready  | control-plane,etcd | 19h | v1.35.4+rke2r1 |
| gpmrke2controlplane2 | Ready  | control-plane,etcd | 19h | v1.35.4+rke2r1 |
| gpmrke2controlplane3 | Ready  | control-plane,etcd | 19h | v1.35.4+rke2r1 |
| gpmrke2worker1       | Ready  | <none>             | 19h | v1.35.4+rke2r1 |
| gpmrke2worker2       | Ready  | <none>             | 19h | v1.35.4+rke2r1 |

# Extra Scenario: Deploy on Kubernetes

- Deploy the Pods
  - Go to the repository folder of `deployment-kubernetes/`
  - Start the rustFS Object Storage by running: `bash deploy-rustfs.sh`
  - Read through [README.md](#) to deploy the environment.
  - You should have this result by the end of tutorial:

```
gpmrke2controlplane1:~ # kubectl get pod -n monitoring
```

| NAME                                        | READY | STATUS  | RESTARTS      | AGE   |
|---------------------------------------------|-------|---------|---------------|-------|
| node-exporter-gzcsx                         | 2/2   | Running | 0             | 13m   |
| node-exporter-rpvgz                         | 2/2   | Running | 0             | 14m   |
| node-exporter-vvt2z                         | 2/2   | Running | 0             | 13m   |
| node-exporter-vzsq2                         | 2/2   | Running | 0             | 14m   |
| node-exporter-xflrw                         | 2/2   | Running | 0             | 14m   |
| prometheus-operator-6f9bf8749d-kvb6t        | 1/1   | Running | 1 (7h21m ago) | 18h   |
| prometheus-prometheus-eu-0                  | 3/3   | Running | 3 (7h21m ago) | 17h   |
| prometheus-prometheus-eu-1                  | 3/3   | Running | 3 (7h21m ago) | 17h   |
| thanos-compact-0                            | 1/1   | Running | 0             | 6h13m |
| thanos-query-prometheus-eu-dbdcd985cd-75vl2 | 1/1   | Running | 0             | 6h25m |
| thanos-query-prometheus-eu-dbdcd985cd-78d2d | 1/1   | Running | 0             | 6h25m |
| thanos-store-0-0                            | 1/1   | Running | 0             | 6h30m |
| thanos-store-1-0                            | 1/1   | Running | 0             | 6h30m |

# Result: Deploy on Kubernetes

Prometheus

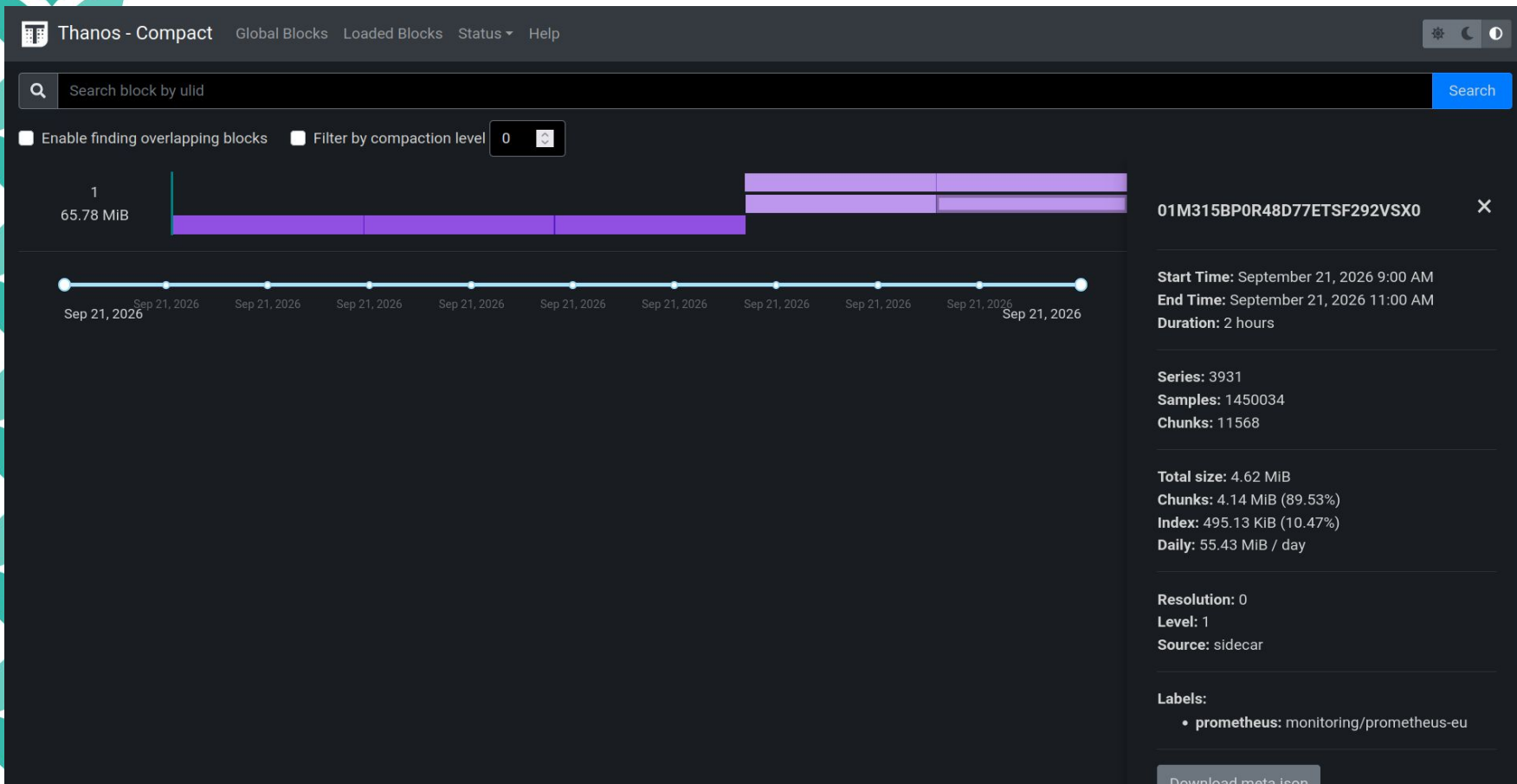
Query Alerts Status > Target health

Select scrape pool Filter by target health Filter by endpoint or labels

serviceMonitor/monitoring/node-exporter/0 5 / 5 up

| Endpoint                             | Labels                                                                                                                                                                                                                                                 | Last scrape                            | State |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-------|
| https://192.168.103.105:9100/metrics | <div>container="kube-rbac-proxy"</div> <div>endpoint="https"</div> <div>instance="gpmrke2worker2"</div> <div>job="node-exporter"</div> <div>namespace="monitoring"</div> <div>pod="node-exporter-nxsc6"</div> <div>service="node-exporter"</div>       | <div>14.025s ago</div> <div>48ms</div> | UP    |
| https://192.168.103.101:9100/metrics | <div>container="kube-rbac-proxy"</div> <div>endpoint="https"</div> <div>instance="gpmrke2controlplane1"</div> <div>job="node-exporter"</div> <div>namespace="monitoring"</div> <div>pod="node-exporter-s7w45"</div> <div>service="node-exporter"</div> | <div>1.5s ago</div> <div>17ms</div>    | UP    |
| https://192.168.103.102:9100/metrics | <div>container="kube-rbac-proxy"</div> <div>endpoint="https"</div> <div>instance="gpmrke2controlplane2"</div> <div>job="node-exporter"</div> <div>namespace="monitoring"</div> <div>pod="node-exporter-bpthw"</div> <div>service="node-exporter"</div> | <div>989ms ago</div> <div>392ms</div>  | UP    |
| https://192.168.103.103:9100/metrics | <div>container="kube-rbac-proxy"</div> <div>endpoint="https"</div> <div>instance="gpmrke2controlplane3"</div> <div>job="node-exporter"</div> <div>namespace="monitoring"</div> <div>pod="node-exporter-2bz88"</div> <div>service="node-exporter"</div> | <div>5.215s ago</div> <div>14ms</div>  | UP    |
| https://192.168.103.104:9100/metrics | <div>container="kube-rbac-proxy"</div> <div>endpoint="https"</div> <div>instance="gpmrke2worker1"</div> <div>job="node-exporter"</div> <div>namespace="monitoring"</div> <div>pod="node-exporter-jw9kf"</div> <div>service="node-exporter"</div>       | <div>11.985s ago</div> <div>16ms</div> | UP    |

# Result: Deploy on Kubernetes



# Result: Deploy on Kubernetes + Scenario 4

Thanos - Query

Graph

Endpoints

Status ▾

Help

Query

show less

| Endpoint             | Status | Announced LabelSets                                                                                             | Min Time (UTC)      | Max Time (UTC) | Last Successful Health Check | Last Message |
|----------------------|--------|-----------------------------------------------------------------------------------------------------------------|---------------------|----------------|------------------------------|--------------|
| 192.168.103.99:32110 | UP     | <code>prometheus="monitoring/prometheus-eu"</code>                                                              | 2026-09-20 18:00:40 | —              | 2.764s ago                   |              |
|                      |        | <code>prometheus="monitoring/prometheus-eu"</code> <code>prometheus_replica="prometheus-prometheus-eu-0"</code> |                     |                |                              |              |
|                      |        | <code>prometheus="monitoring/prometheus-eu"</code> <code>prometheus_replica="prometheus-prometheus-eu-1"</code> |                     |                |                              |              |

Sidecar

show less

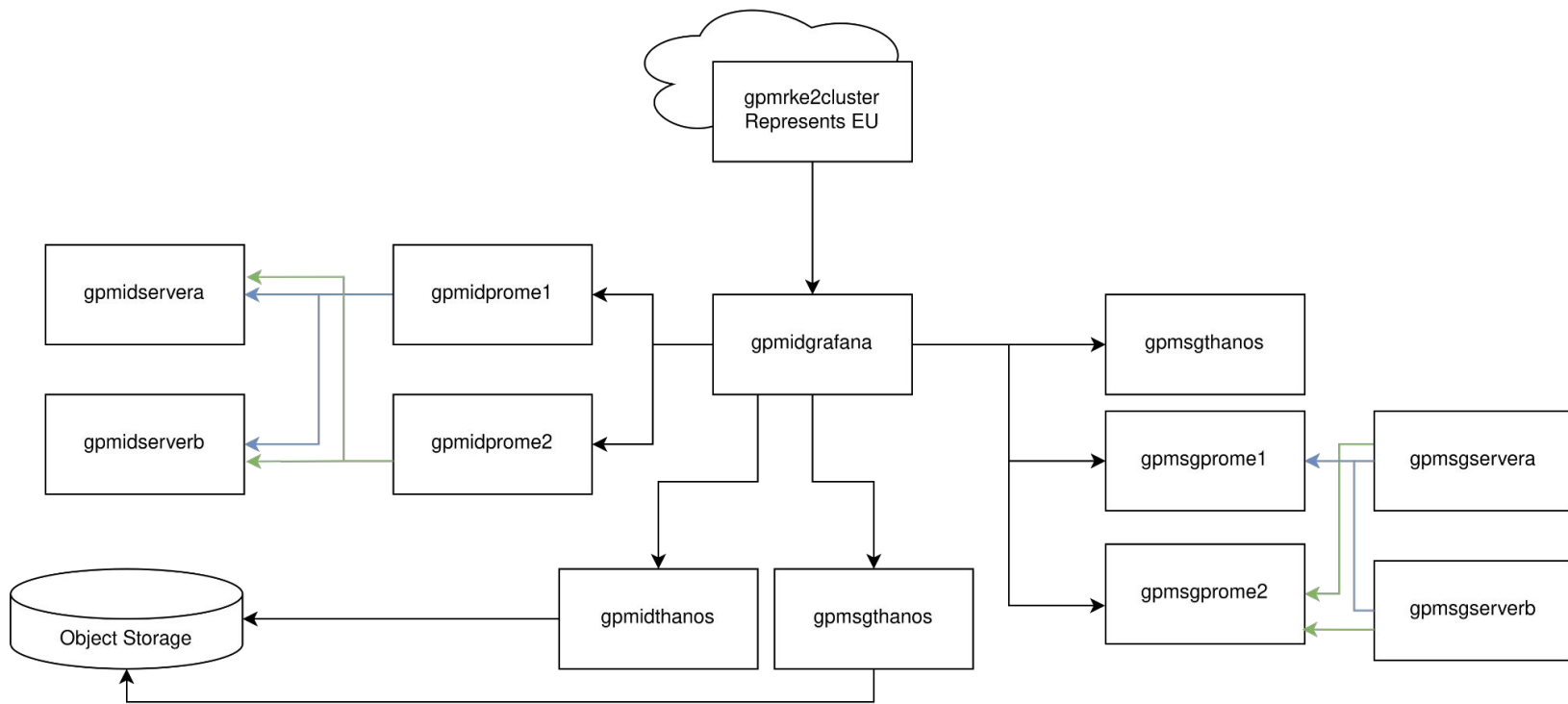
| Endpoint         | Status | Announced LabelSets                                        | Min Time (UTC)      | Max Time (UTC) | Last Successful Health Check | Last Message |
|------------------|--------|------------------------------------------------------------|---------------------|----------------|------------------------------|--------------|
| gpmidprom1:10900 | UP     | <code>data_center="id"</code> <code>replica="prom1"</code> | 2026-09-20 16:22:08 | —              | 2.766s ago                   |              |
| gpmidprom2:10900 | UP     | <code>data_center="id"</code> <code>replica="prom2"</code> | 2026-09-20 16:22:16 | —              | 2.765s ago                   |              |
| gpmsgprom1:10900 | UP     | <code>data_center="sg"</code> <code>replica="prom1"</code> | 2026-09-20 16:22:29 | —              | 2.767s ago                   |              |
| gpmsgprom2:10900 | UP     | <code>data_center="sg"</code> <code>replica="prom2"</code> | 2026-09-20 16:22:26 | —              | 2.767s ago                   |              |

Store

show less

| Endpoint          | Status | Announced LabelSets                                        | Min Time (UTC)      | Max Time (UTC)      | Last Successful Health Check | Last Message |
|-------------------|--------|------------------------------------------------------------|---------------------|---------------------|------------------------------|--------------|
| gpmidthanos:10902 | UP     | <code>data_center="id"</code>                              | 2026-09-08 12:14:16 | 2026-09-21 04:00:00 | 2.769s ago                   |              |
|                   |        | <code>data_center="id"</code> <code>replica="prom1"</code> |                     |                     |                              |              |
|                   |        | <code>data_center="id"</code> <code>replica="prom2"</code> |                     |                     |                              |              |
| gpmsgthanos:10902 | UP     | <code>data_center="sg"</code>                              | 2026-09-08 14:40:35 | 2026-09-21 04:00:00 | 2.771s ago                   |              |
|                   |        | <code>data_center="sg"</code> <code>replica="prom1"</code> |                     |                     |                              |              |

# Result: Deploy on Kubernetes + Scenario 4



# OOT: My Current Memory Usage :v



**Thank You**



**openSUSE®**  
Asia Summit 2026